Running CETP

Step-by-step, from a fresh clone to reproducing every published number. If you only read one section, read **§2**.

---

0. Prerequisites

	Needed for	Check
Python 3.13	everything	`python3 --version`
PostgreSQL 14+ client and server	measuring machines	`psql --version`
Docker	the capability grid only	`docker --version`
Chrome/Chromium	rebuilding the PDFs only	—

Python 3.13 specifically: `requirements.txt` pins `numpy==2.5.3`, which has no wheel for 3.11. The shipped `cetp` package itself needs only the standard library, so **3.11+ is fine if you are just using the tool** — the pin matters for the research scripts.

---

1. Install

```
git clone https://github.com/harshithjn/CETP_v1.git
cd CETP_v1
pip install -e .
```

That is enough to use the tool. **It has no runtime dependencies.**

For the research scripts (evaluations, the grid, retraining):

```
pip install -r requirements.txt
```

---

2. Verify it works

```
cetp selftest
```

Expected:

```
redaction             OK
prediction            OK  (3587 ms dev -> 2122 ms prod, scaling 0.592)
regression (identity) OK  (same plan twice -> ratio 1.000, hardware term cancels)
calibration           OK
operator model        OK  (identical machines 1.000; prod without parallelism 2.50x slower)

selftest PASSED
```

Then the full suite:

```
make test        # 108 tests
```

If both pass, the installation is good.

---

3. Use it — predicting a query's production runtime

3.1 Measure each machine once

```
# on your development machine
python3 scripts/collection/operator_probe.py --machine-id dev-laptop --db mydb

# on production (or pointing at it)
python3 scripts/collection/operator_probe.py --machine-id prod-db --db mydb \
    --host prod.example.com --user readonly
```

Takes a few minutes per machine. It creates two temporary tables, times six operations, and drops them.

> **Run this on an idle machine.** Under load it measures contention rather than capability — observed moving a machine's measured parallel speedup from 2.47× to 1.14×. The probe detects this and flags the record unreliable, and the predictor then refuses to use it.

It works on **managed PostgreSQL** (RDS, Aurora, Cloud SQL) because it is only SQL. A read-only user is enough, plus permission to create two temporary tables.

Check what is registered. Note these are two different registries:

```
# the OPERATOR signatures, which predict-prod uses
python3 -c "import json;r=json.load(open('models/operator_signature.json'));\
[print(f\"{m:20s} {v['client_platform']:16s} reliable={v['reliable']}\") for m,v in
sorted(r.items())]"

# the LEGACY two-scalar registry, which `cetp predict` and the VS Code extension use
cetp signature list
```

3.2 Predict

```
python3 scripts/collection/predict_prod.py --query-file q.sql \
    --dev dev-laptop --prod prod-db \
    --prod-host prod.example.com --prod-db mydb --prod-user readonly \
    --dev-db mydb --sla-ms 2000
```

What happens: production is `EXPLAIN`ed (**planned, never executed**), that plan is forced on your machine with `pg_hint_plan` and measured there, and the result is scaled by the two capability signatures.

Read the `trust:` line and the warnings. They are the difference between a usable number and a misleading one.

3.3 If the two machines have different CPU architectures

The prediction is **refused**:

```
REFUSED: refusing an uncalibrated prediction from arm64 to x86_64: measured at
72.84% MAPE against 21.41% for assuming production equals development...
```

This is deliberate. Uncalibrated cross-architecture prediction was measured as roughly 3× worse than doing nothing, on every x86 machine tried. To lift it, measure three to five queries on production and feed the real numbers back:

```
python3 scripts/collection/predict_prod.py --query-file q1.sql \
    --dev dev-laptop --prod prod-db ... --actual-ms 1840
```

Each `--actual-ms` records an observation. After three the correction stops being provisional; after five the refusal lifts and accuracy is near the best achievable for that pair.

---

4. Use it — regression checking (the most reliable mode)

Comparing the *same* query before and after a change is the mode to trust: the hardware term appears in both numerator and denominator and largely cancels.

```
psql -d mydb -tAc "EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) $(cat before.sql)" > before.json
psql -d mydb -tAc "EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) $(cat after.sql)"  > after.json

cetp regress --before before.json --after after.json \
             --dev dev-laptop --prod prod-db --threshold 1.20 --fail-on-regression
```

`--fail-on-regression` exits non-zero, which is all CI needs.

---

5. Before any plan leaves your machine

`EXPLAIN` output embeds your query's literal values. On a real database those are user data.

```
cetp redact plan.json -o plan.safe.json          # strip literals, keep structure
cetp redact plan.json -o plan.pub.json --level full   # also hash identifiers
cetp redact plan.safe.json --check               # verify; exits 1 if anything survived
```

---

6. Reproduce the research results

6.1 The legacy five-machine dataset

```
make results            # writes results/CANONICAL.md + canonical_metrics.json
make verify-determinism # runs it twice, asserts byte-identical output
```

6.2 The capability grid (needs Docker, ~3 hours)

```
make grid-setup      # load TPC-H SF1 into a Docker volume. Once. ~10 min, ~2 GB
make grid-collect    # sweep 4 CPU x 2 memory configurations. ~2-3 h
make operator-results
```

`make grid-reprobe` re-measures capability without redoing the collection.

6.3 The rest

```
python3 scripts/analysis/laptop_to_prod_evaluation.py   # the real scenario
python3 scripts/analysis/laptop_to_ec2_evaluation.py    # cross-ISA on cloud hardware
python3 scripts/analysis/concurrency_model.py           # elasticity
python3 scripts/analysis/version_compat_check.py        # parser vs PG 14/15/17/18
```

Each writes a `.md` and a `.json` into `results/`.

6.4 Collecting a free x86 machine

Push to the `x86-probe` branch, or dispatch `.github/workflows/x86-operator-probe.yml`. It loads TPC-H on a GitHub Actions runner, probes it, collects 22 queries and uploads the results as an artifact. Download with `gh run download <id> -n x86-probe-results-ubuntu-latest`, copy the CSV into `data/container_grid/` and merge the signature into `models/operator_signature.json`.

---

7. Rebuild the documents and the site

```
make docs        # docs/LLD.pdf and docs/HANDBOOK.pdf, from the Markdown
open site/index.html
```

`site/index.html` is one self-contained file with no external requests. Host it by uploading that single file.

---

8. Troubleshooting

Symptom	Cause and fix
`signature is flagged unreliable`	The machine was busy when probed. Stop other work and re-probe.
`refusing an uncalibrated prediction from X to Y`	Different CPU architectures. See §3.3.
`probe scales differ, rates are not comparable`	The two machines were probed with different probe settings. Re-probe both with the current script.
`carries operator_rates_v1`	A signature from before the parallelism axis existed. Re-probe.
`ModuleNotFoundError: No module named '_loss'`	scikit-learn version mismatch against the pickled models. `pip install -r requirements.txt`.
`Could not find a version that satisfies numpy==2.5.3`	You are on Python 3.11. Use 3.13 for the research scripts.
Gate exits 1 in CI	That is a **BLOCK verdict**, not a failure. 0 = pass/warn, 1 = block, 2+ = the gate itself broke.
`pg_hint_plan is not available`	Plan forcing needs it on the **development** machine only. Without it, pass `--no-force`; the result is reported with `plan_forced=False` and lower trust.
Probe hangs on a managed instance	It needs permission to create two temporary tables. Check the role.

9. What to read next

`docs/HANDBOOK.md` / `.pdf`	A-to-Z: concepts, findings, history, limits. **Start here.**
`docs/LLD.md` / `.pdf`	The design document
`results/*.md`	Every number, generated. Never hand-edited
`site/index.html`	The public summary
`TASKS.md`	Full history and open work